Python-Pydantic-M-Set1

Medium | Code behavior and data contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

01. An evidence collector is called twice in the same process. Each call is intended to start a fresh list. What does the second print show, and which change fixes that default behavior?

```
def collect(doc_id, ids=[]):
    ids.append(doc_id)
    return ids
print(collect("A"))
print(collect("B"))
```

A. ['B']; the existing default already creates a list per call.

B. ['A', 'B']; default to None and create a list inside the function.

C. ['A', 'B']; copy ids only when returning from the function.

D. ['A']; replace append with extend([doc_id]) inside the function.

02. A request copies a template before changing its filters. What prints, and what is the smallest change that isolates the tags list in this exact two-level structure?

```
template = {"tags": ["base"]}
request = template.copy()
request["tags"].append("urgent")
print(template["tags"])
```

A. ['base', 'urgent']; replace template.copy() with dict(template).

B. ['base']; dict.copy() already isolates the nested list.

C. ['base', 'urgent']; assign request['tags'] = template['tags'].copy() before append.

D. ['base']; replacing append with += ['urgent'] makes the operation private.

Python-Pydantic-M-Set1

Medium | Code behavior and data contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

03. The cache contains the key. What prints, and which replacement avoids running fetch() on this cache hit while preserving a cached None value?

```
cache = {"q": 7}
key = "q"
def fetch():
    print("fetch")
    return 9
print(cache.get(key, fetch()))
```

A. fetch then 7; use cache[key] if key in cache else fetch().

B. 7 only; dict.get evaluates its default only on a miss.

C. fetch then 7; use cache.get(key) or fetch().

D. fetch then 9; the evaluated default overwrites the cached value.

---

04. All input IDs are positive. What does persist receive, and which fix preserves both complete validation and the same records for this small in-memory batch?

```
rows = ({"id": i} for i in [1, 2, 3])
assert all(row["id"] > 0 for row in rows)
persist(list(rows))
```

A. An empty list; call iter(rows) again before persisting.

B. All three rows; all() inspects a generator without advancing it.

C. Only the last row; all() retains its last successful item.

D. An empty list; materialize once, then validate and persist that list.

Python-Pydantic-M-Set1

Medium | Code behavior and data contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

05. Three callbacks are registered for three document partitions. What prints, and which edit makes each callback retain its own partition name?

```
callbacks = []
for name in ["A", "B", "C"]:
    callbacks.append(lambda: name)
print([fn() for fn in callbacks])
```

A. ['C', 'C', 'C']; use lambda name=name: name.

B. ['A', 'B', 'C']; loop variables are captured by value.

C. ['A', 'A', 'A']; reverse the names before registration.

D. ['C', 'C', 'C']; rename the loop variable to partition.

---

06. The embedding service silently returned only two vectors for three IDs. This batch must fail before any database write if counts differ. Which replacement for pairs enforces that requirement?

```
ids = ["d1", "d2", "d3"]
vectors = [[0.1], [0.2]]
pairs = list(zip(ids, vectors))
write(pairs)
```

A. pairs = zip(ids, vectors, strict=True)

B. pairs = list(zip(ids, vectors))

C. pairs = list(zip(ids, vectors, strict=True))

D. pairs = list(zip(ids[:len(vectors)], vectors))

Python-Pydantic-M-Set1

Medium | Code behavior and data contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

07. A webhook deserializes JSON into a plain dictionary. What happens at the shown access, and what is the appropriate runtime boundary?

```
class Request(TypedDict):
    query: str
payload: Request = {}
print(payload["query"])
```

A. An empty string is inserted; require a nonempty string downstream.

B. A typing validation error at assignment; catch it around the annotation.

C. None is returned; make downstream code handle an optional value.

D. KeyError; validate the payload before accessing the required field.

---

08. Assume send() raises ConnectionError. What does the caller observe, and which change preserves the failure while still allowing cleanup?

```
def submit():
    try:
        send()
        return "ok"
    finally:
        return "done"
```

A. ConnectionError; finally cannot affect a pending exception.

B. The string 'done'; remove return from finally and perform cleanup there.

C. The string 'ok'; the earlier return runs before the exception.

D. ConnectionError; moving return 'done' to except preserves it.

Python-Pydantic-M-Set1

Medium | Code behavior and data contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

09. The public service layer must raise RetrievalUnavailable while retaining the lower-level TimeoutError as its explicit cause for diagnostics. Which handler meets both requirements?

A. except TimeoutError as exc: raise exc

B. except TimeoutError as exc: raise RetrievalUnavailable("search failed") from None

C. except TimeoutError as exc: return RetrievalUnavailable("search failed")

D. except TimeoutError as exc: raise RetrievalUnavailable("search failed") from exc

---

10. The caller consumes the returned iterator only after lines() returns. The file exists and contains text. What is the defect, and which change preserves lazy reading correctly?

```
def lines(path):
    with open(path) as f:
        return (line.strip() for line in f)
result = lines(path)
print(next(result))
```

A. No defect; the generator expression keeps the file open automatically.

B. Iteration uses a closed file; yield stripped lines inside the with block.

C. The file stays open forever; call close() immediately before return.

D. Iteration starts at EOF; call seek(0) immediately before return.

Python-Pydantic-M-Set1

Medium | Code behavior and data contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

11. A response contract requires the reason key, but permits JSON null. Under Pydantic 2 defaults, which statement is true?

```
class Result(BaseModel):
    reason: str | None
```

A. Result() fails, while Result(reason=None) succeeds; the declaration matches the contract.

B. Both calls succeed; str | None automatically supplies a None default.

C. Both calls fail; unions containing None cannot be model fields.

D. Only Result() succeeds; explicit None is treated as a missing required value.

---

12. A tool contract requires a real integer retry count, not a boolean or a numeric string. Which inputs succeed for this model: 3, "3", and True?

```
class RunConfig(BaseModel):
    retries: StrictInt
```

A. 3 and True succeed.

B. 3 and "3" succeed.

C. Only 3 succeeds.

D. All three succeed.

Python-Pydantic-M-Set1

Medium | Code behavior and data contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

13. Constructing Limits() currently succeeds with budget=-1. What is the smallest change that makes this invalid default fail validation while preserving the positive-budget constraint?

```
class Limits(BaseModel):
    budget: int = Field(default=-1, gt=0)
```

A. Change gt=0 to ge=0 only.

B. Add strict=True to Field(...) only.

C. Add validate_default=True to Field(...).

D. Use model_construct() instead of Limits().

---

14. RunConfig.model_validate({"max_retires": 0}) succeeds but max_retries remains 3. The service should reject misspelled request fields. Which change best enforces that policy?

```
class RunConfig(BaseModel):
    max_retries: int = 3
```

A. Set model_config = ConfigDict(extra="allow").

B. Set model_config = ConfigDict(extra="forbid").

C. Set model_config = ConfigDict(strict=True) only.

D. Make max_retries nullable while keeping its default.

Python-Pydantic-M-Set1

Medium | Code behavior and data contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

15. An upstream client sends either one numeric string or a list of numeric strings. What does Batch(ids="7").ids become, and what happens for Batch(ids=["7", "bad"])?

```
class Batch(BaseModel):
    ids: list[int]
    @field_validator("ids", mode="before")
    @classmethod
    def as_list(cls, value):
        return value if isinstance(value, list) else [value]
```

A. [7]; the mixed list fails integer-item validation.

B. ["7"]; a before validator disables all later type validation.

C. [7]; the mixed list is silently reduced to [7].

D. The scalar input fails; before validators run only after list validation.

---

16. A window must satisfy end > start. Both inputs should first be parsed as integers, and the check should not depend on their declaration order. Which implementation best meets these constraints?

```
class Window(BaseModel):
    start: int
    end: int
```

A. A field constraint gt=0 on end guarantees the ordering for every start.

B. A before field validator on end compares the raw value with info.data["start"].

C. A serializer on end rejects the window only when model_dump is called.

D. An after model validator compares self.end and self.start, raises ValueError on failure, and returns self.

Python-Pydantic-M-Set1

Medium | Code behavior and data contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

17. An HTTP client will JSON-encode the supplied dictionary itself. It requires a dictionary containing only JSON-compatible values. Which expression is appropriate for this model?

```
class Payload(BaseModel):
    at: datetime
    amount: Decimal
payload = Payload(at=datetime(2026, 1, 1),
                  amount=Decimal("12.50"))
```

A. json.dumps(payload.model_dump()) with no custom encoder

B. payload.model_dump()

C. payload.model_dump_json()

D. payload.model_dump(mode="json")

18. For this PATCH contract, omitting note means leave it unchanged; sending note=null means clear it. Which expression preserves that distinction for both a and b?

```
class Patch(BaseModel):
    note: str | None = None
a = Patch()
b = Patch(note=None)
```

A. model_dump(): a gives {}, b gives {"note": None}.

B. model_dump(exclude_none=True): a gives {}, b gives {"note": None}.

C. model_dump(exclude_unset=True): a gives {}, b gives {"note": None}.

D. model_dump(exclude_defaults=True): a gives {}, b gives {"note": None}.

Python-Pydantic-M-Set1

Medium | Code behavior and data contracts | 20 MCQs

Choose one best answer. Assume Python 3.12 and Pydantic 2. Each snippet is independent; standard imports and stated setup are available.

19. After validating the supplied external key, which result does model_dump(by_alias=True) produce?

```
class Event(BaseModel):
    request_id: str = Field(
        validation_alias="request-id",
        serialization_alias="requestId")
e = Event.model_validate({"request-id": "r1"})
print(e.model_dump(by_alias=True))
```

A. {"request_id": "r1"}

B. {"requestId": "r1"}

C. {"request-id": "r1"}

D. {"request-id": "r1", "requestId": "r1"}

20. The second item has an invalid qty. Ignoring error-message wording, which loc tuple identifies that failure in ValidationError.errors()?

```
class Item(BaseModel):
    qty: int
class Batch(BaseModel):
    items: list[Item]
Batch.model_validate({"items": [{"qty": 1}, {"qty": "bad"}]})
```

A. ('items', 1, 'qty')

B. ('items', 2, 'qty')

C. ('qty',)

D. ('items', 'qty', 'bad')